Quantum vs Classical Stock Prediction System

By: Hastons Isiavwe

Stock_Data.py Code

```
1  import random
2
   1 usage
3  def generate_stock_data(output_file="stock_data.txt", days=365):
4      """
```

This snippet contains the initial lines of the `stock_Data.py` script, setting up the environment for generating random stock data. Here's a detailed explanation:

Line 1:

import random:

- The random library is imported to facilitate the generation of random floating-point numbers. This is essential for creating daily percentage changes for the stock data simulation.
- **Purpose**: To generate random stock price changes, which simulate the fluctuation in stock values.

Line 3 – 4:

 def generate_stock_data:

- **Function Definition**: This defines a reusable function called generate_stock_data that will handle the generation and saving of stock data.
- **Parameters**:
  - output_file: Specifies the file name where the generated data will be stored. Default is "stock_data.txt".
  - days: The number of days for which stock price changes will be generated. Default is 365, representing one year.

 Docstring:

- The triple-quoted text explains the purpose of the function and its parameters.
- **Purpose**: To document what the function does, aiding in code readability and maintenance.

```
11      percentage_changes = [
12          round(random.uniform(-100.00, b: 100.00), 2) for _ in range(days)
13      ]
14
15      # Write to file
16      with open(output_file, 'w') as file:
17          file.writelines(f"{change:+.2f}\n" for change in percentage_changes)
18
19      print(f"Generated stock data written to '{output_file}'.")
20      return percentage_changes
21
```

This section of the code is the implementation of the generate_stock_data function, which generates daily stock percentage changes and writes them to a file.

Line 11 – 13:

percentage_changes:

- This is a **list comprehension** used to generate a list of random percentage changes.
- **random.uniform(-100.00, 100.00)**:
  - Generates a random floating-point number between -100.00 and 100.00, simulating percentage changes ranging from -100% to +100%.
- **round(..., 2)**:
  - Rounds each generated value to 2 decimal places for clarity and realistic representation.
- **for _ in range(days)**:
  - Repeats the random generation process for the specified number of days.
  - _ is a placeholder variable, indicating that the loop variable is unused.

Line 16 – 17:

File Writing:

- **with open(output_file, 'w') as file:**:
  - Opens the file specified by output_file (default: "stock_data.txt") in write mode ('w').
  - Ensures the file is properly closed after the operation is completed.
- **file.writelines(...)**:
  - Writes the generated percentage changes to the file.
  - **f'{change:+.2f}\n'**:
    - Formats each change to include a + or - sign, 2 decimal places, and adds a newline character (\n).

        - Example: +12.34 or -56.78.
    - **for change in percentage_changes**:
        - Iterates through the list percentage_changes, writing each formatted value to the file.

Line 19n – 20:

Feedback and Return:

- **print(...)**:
    - Outputs a message indicating the data has been written to the file, providing confirmation to the user.
- **return percentage_changes**:
    - Returns the list of percentage changes for potential further use in the program.

Classical_P.py Code

```
1  def simulate_stock_performance(input_file="stock_data.txt", initial_price=200, budget=10_000):
2      """
```

Line 1:

Code Breakdown

1. **def simulate_stock_performance(...):**
    - This defines a Python function called simulate_stock_performance.
2. **input_file="stock_data.txt":**
    - The function takes an optional argument input_file with a default value of "stock_data.txt".
    - This argument is likely used to specify a file containing stock data for the simulation.
3. **initial_price=200:**
    - Another optional argument, initial_price, is set to 200 by default.
    - This is likely the starting price of the stock for the simulation.
4. **budget=10_000:**
    - The budget argument specifies the initial budget for the stock simulation.
    - The default value is 10,000, written using an underscore for better readability.

```
11    with open(input_file, 'r') as file:
12        percentage_changes = [float(line.strip()) for line in file.readlines()]
13
14    shares_owned = 0
15    results = []
16
```

Line 11 – 15

This code snippet reads data from a file, processes it into percentage changes, and sets up variables for a stock simulation. Here's the explanation in your own words:

Code Explanation

1. **with open(input_file, 'r') as file:**
    - This opens the file specified by input_file in read mode ('r').
    - The with statement ensures the file is properly closed after reading, even if an error occurs.
2. **percentage_changes = [...]**
    - This creates a list comprehension that processes each line in the file.
    - **What it does:**
        - file.readlines() reads all lines in the file.
        - line.strip() removes any extra spaces or newline characters from each line.
        - float(...) converts the cleaned line into a floating-point number.
    - The resulting percentage_changes list contains all the percentage changes from the file as floats.
3. **shares_owned = 0**
    - Initializes the variable shares_owned to 0.
    - This likely tracks the number of shares owned during the stock simulation.
4. **results = []**
    - Initializes an empty list called results.
    - This is likely used to store the results or key metrics of the simulation.

```
18    for day in range(0, len(percentage_changes), 90):
19        # Calculate average percentage change over the window
20        window = percentage_changes[day:day + 90]
21        avg_change = sum(window) / len(window)
22        predicted_price = initial_price * (1 + avg_change / 100)
23
```

This code snippet calculates predicted stock prices based on average percentage changes over a 90-day window.

Line 18 – 22:

Code Breakdown

1. **for day in range(0, len(percentage_changes), 90):**
    - Loops through the percentage_changes list in steps of 90 days.
    - The range starts at 0 and goes up to the length of the percentage_changes list.
    - Each loop iteration processes a 90-day window of percentage changes.
2. **window = percentage_changes[day:day + 90]**
    - Extracts a 90-day "window" of percentage changes from the list.
    - The slicing (day:day + 90) selects elements starting from day and ending at day + 90.
3. **avg_change = sum(window) / len(window)**
    - Calculates the average percentage change over the 90-day window.
    - Adds up all the percentage changes in the window using sum() and divides by the number of days (len(window)).
4. **predicted_price = initial_price * (1 + avg_change / 100)**
    - Uses the average percentage change to calculate the predicted stock price.
    - **Formula Explanation**:
        - initial_price is multiplied by (1 + avg_change / 100) to account for the percentage change.
        - For example, if avg_change is 5, the predicted price will be initial_price * 1.05 (5% increase).

```
25        if predicted_price <= 67:
26            shares_to_buy = 0
27        elif predicted_price <= 200:
28            shares_to_buy = 150
29        else:
30            shares_to_buy = 50
31
32        cost = shares_to_buy * predicted_price
33
```

Line 25 – 32:

This code snippet determines how many shares to buy based on the predicted stock price and calculates the total cost. Here's the explanation in your own words:

Code Breakdown

1. **if predicted_price <= 67:**
    - Checks if the predicted price is less than or equal to $67.
    - **Action:** Sets shares_to_buy to 0, meaning no shares will be purchased if the price is too low (perhaps considered risky or unfavorable).
2. **elif predicted_price <= 200:**
    - Checks if the predicted price is between $68 and $200 (inclusive).
    - **Action:** Sets shares_to_buy to 150. This means 150 shares will be purchased when the stock price is within this range.
3. **else:**
    - Covers all cases where the predicted price is greater than $200.
    - **Action:** Sets shares_to_buy to 50. This means fewer shares are purchased when the stock price is high, possibly due to a higher cost or lower expected returns.
4. **cost = shares_to_buy * predicted_price**
    - Calculates the total cost of buying the shares.
    - **Formula Explanation:** Multiplies the number of shares to buy (shares_to_buy) by the predicted price of the stock (predicted_price).

```
35        if cost > budget:
36            affordable_shares = int(budget // predicted_price)
37            shares_to_buy = min(shares_to_buy, affordable_shares)
38            cost = shares_to_buy * predicted_price
39
40        if shares_to_buy > 0:
41            budget -= cost
42            shares_owned += shares_to_buy
43            results.append(f"Day {day + 90}: Bought {shares_to_buy} shares at ${predicted_price:.2f}. Remaining budget: ${budget:.2f}")
44        else:
45            results.append(f"Day {day + 90}: Not enough budget to buy shares. Predicted Price: ${predicted_price:.2f}.")
46
47        initial_price = predicted_price
48
```

Line 35 – 47:

This code snippet manages the logic for purchasing shares within a budget and updates the simulation results.

Code Breakdown

1. **if cost > budget:**
    - Checks if the total cost of purchasing the desired shares (cost) exceeds the available budget.
    - If true, adjusts the number of shares to buy to fit within the budget.
2. **affordable_shares = int(budget // predicted_price)**

- Calculates the maximum number of shares that can be afforded with the current budget.
- Uses integer division (//) to ensure the result is a whole number.

3. **shares_to_buy = min(shares_to_buy, affordable_shares)**
   - Ensures the number of shares to buy is the lesser of the desired shares (shares_to_buy) or the number that can be afforded (affordable_shares).
4. **cost = shares_to_buy * predicted_price**
   - Recalculates the cost of purchasing the adjusted number of shares.
5. **if shares_to_buy > 0:**
   - Checks if any shares are being bought. If true:
     - **budget -= cost**: Deducts the cost of the purchase from the budget.
     - **shares_owned += shares_to_buy**: Adds the purchased shares to the total shares owned.
     - **Logs the purchase in the results list** with details like day, number of shares bought, predicted price, and remaining budget.
6. **else:**
   - If no shares are purchased (e.g., budget is insufficient or shares_to_buy is 0):
     - Logs a message in the results list indicating insufficient budget to buy shares, along with the predicted price
7. **initial_price = predicted_price**
   - Updates the initial_price to the predicted_price for the next iteration of the simulation.
   - This sets up the next cycle with the latest stock price.

```
50      final_value = shares_owned * initial_price + budget
51      results.append(f"Final Portfolio Value: ${final_value:.2f}")
52      print("\n".join(results))
53      return final_value
54
```

Line 50 – 53:

This code snippet calculates the final portfolio value at the end of the simulation and logs the result.

Code Breakdown

1. **final_value = shares_owned * initial_price + budget**
   - Calculates the total value of the portfolio at the end of the simulation.

- **Formula Explanation:**
  - shares_owned * initial_price: Calculates the total value of the shares currently owned at the latest stock price (initial_price).
  - + budget: Adds the remaining budget to include unused funds.

2. **results.append(f"Final Portfolio Value: ${final_value:.2f}")**
   - Appends a formatted string to the results list with the final portfolio value.
   - **Formatting Explanation:** ${final_value:.2f} ensures the value is displayed as a monetary amount with two decimal places.
3. **print("\n".join(results))**
   - Prints all the logged results (stored in the results list) to the console.
   - "\n".join(results) combines all entries in the results list into a single string, separating them by new lines (\n).
4. **return final_value**
   - Returns the calculated final portfolio value as the output of the function.

QuantumCircuit.py Code

This section of the code contains the **import statements** required for the quantum computing-based implementation of the stock prediction task. These libraries provide the tools needed for creating quantum circuits, running simulations, visualizing results, and plotting data.

QuantumCircuit.py × Quantum_Circuit_With_All.py Classical_P.py

```
1  from qiskit import QuantumCircuit, transpile
2  from qiskit_aer import AerSimulator
3  from qiskit.visualization import plot_histogram
4  import matplotlib.pyplot as plt
5
6
```

Line 1:

1. from qiskit import QuantumCircuit, transpile

- **QuantumCircuit**:
  - This is a core class in the Qiskit library used to define and manipulate quantum circuits.
  - A quantum circuit is a series of quantum operations (gates) applied to qubits.

    - Example Usage: circuit = QuantumCircuit(num_qubits) creates a quantum circuit with a specified number of qubits.

- **transpile**:
    - Optimizes quantum circuits for specific quantum hardware or simulators.
    - Converts the circuit into a lower-level form compatible with the backend (quantum hardware or simulator).
    - Example Usage: compiled_circuit = transpile(circuit, simulator).

Line 2:

2. from qiskit_aer import AerSimulator

- **AerSimulator**:
    - Part of Qiskit's Aer module, used to simulate quantum circuits on classical computers.
    - Provides an efficient way to test quantum circuits without requiring real quantum hardware.
    - Example Usage: simulator = AerSimulator() initializes the simulator.

Line 3:

This section of the code contains the **import statements** required for the quantum computing-based implementation of the stock prediction task. These libraries provide the tools needed for creating quantum circuits, running simulations, visualizing results, and plotting data.

Code Breakdown

1. from qiskit import QuantumCircuit, transpile

- **QuantumCircuit**:
    - This is a core class in the Qiskit library used to define and manipulate quantum circuits.
    - A quantum circuit is a series of quantum operations (gates) applied to qubits.
    - Example Usage: circuit = QuantumCircuit(num_qubits) creates a quantum circuit with a specified number of qubits.

- **transpile**:
    - Optimizes quantum circuits for specific quantum hardware or simulators.
    - Converts the circuit into a lower-level form compatible with the backend (quantum hardware or simulator).
    - Example Usage: compiled_circuit = transpile(circuit, simulator).

---

2. from qiskit_aer import AerSimulator

- **AerSimulator**:
  - Part of Qiskit's Aer module, used to simulate quantum circuits on classical computers.
  - Provides an efficient way to test quantum circuits without requiring real quantum hardware.
  - Example Usage: simulator = AerSimulator() initializes the simulator.

---

3. from qiskit.visualization import plot_histogram

- **plot_histogram**:
  - A function to visualize the results of quantum circuit executions (measurements) as a histogram.
  - The histogram shows the probabilities of different measurement outcomes (binary strings).
  - Example Usage: plot_histogram(counts) displays the measurement outcomes.

Line 4:

4. import matplotlib.pyplot as plt

- **matplotlib.pyplot**:
  - A Python library for creating static, interactive, and animated visualizations.
  - Here, it is used for customizing and saving plots generated by Qiskit.
  - Example Usage:
    - plt.title("Quantum Circuit Diagram") adds a title to a plot.
    - plt.savefig("filename.png") saves the plot as an image.

```
7   def generate_quantum_stock_data(num_days=365, num_qubits=9):
8       """
```

Line 7: def generate_quantum_stock_data(num_days=365, num_qubits=9):

def: Defines the function.

generate_quantum_stock_data: The function's name, indicating it generates quantum-inspired stock data.

Parameters:

- **num_days**:
    - Default value: 365.
    - Specifies the number of days for which stock data will be simulated.
    - Example Usage: If num_days=100, the function generates data for 100 days.
- **num_qubits**:
    - Default value: 9.
    - Specifies the number of qubits in the quantum circuit. Each qubit contributes to creating a quantum state that determines the stock data.

```
15        circuit = QuantumCircuit(num_qubits)
16
```

Line 15: circuit = QuantumCircuit(num_qubits)

 QuantumCircuit: Initializes a quantum circuit with the specified number of qubits.

 Purpose:

- A quantum circuit is the foundation for applying quantum gates and simulating quantum behavior.
- Here, the circuit is used to generate random states (basis for percentage changes).

 Example:

- If num_qubits=9, the circuit will have 9 qubits, allowing 2^9=512 potential quantum states.

```
18        for qubit in range(num_qubits):
19            circuit.h(qubit)
20
```

Line 18 – 19:

This snippet of code applies a Hadamard gate to all qubits in a quantum circuit using a loop. Here's a breakdown:

1. **for qubit in range(num_qubits):**
    - This iterates over all qubits in the quantum circuit.
    - num_qubits is expected to be the total number of qubits in the circuit.
2. **circuit.h(qubit)**
    - This applies the Hadamard gate (h) to the qubit specified by the qubit variable.
    - The Hadamard gate creates a superposition of states for the qubit.

- In Qiskit, this is typically used to prepare a uniform superposition state.

```
22        for qubit in range(num_qubits - 1):
23            circuit.cx(qubit, qubit + 1)
24
```

Line 22 – 23:

This code applies a controlled NOT (CNOT) gate between neighboring qubits in a quantum circuit. Here's a detailed explanation:

Code Breakdown

1. **for qubit in range(num_qubits - 1):**
    - This loop iterates over all qubits except the last one.
    - range(num_qubits - 1) ensures that the last qubit is not used as the control qubit since there's no "next" qubit for it.
2. **circuit.cx(qubit, qubit + 1)**
    - The CNOT gate (cx) is applied.
    - qubit acts as the control qubit, and qubit + 1 acts as the target qubit.
    - A CNOT gate flips the target qubit's state if the control qubit is in the $|1\rangle$ state.

```
29        simulator = AerSimulator()
30        compiled_circuit = transpile(circuit, simulator)
31        result = simulator.run(compiled_circuit, shots=num_days).result()
32        counts = result.get_counts()
33
```

This code runs a quantum circuit on a simulator multiple times, gathers the results, and counts how often each outcome occurs. It's a standard process in quantum computing to analyze the probabilistic behavior of quantum circuits.

Line 29 – 32:

simulator = AerSimulator()

- This initializes a quantum simulator from Qiskit's Aer library.
- The simulator is used to mimic the behavior of a real quantum computer without needing actual hardware.

2. **compiled_circuit = transpile(circuit, simulator)**
    - The transpile function optimizes the quantum circuit to be run on the simulator.
    - This step adapts the circuit to match the simulator's requirements and capabilities.

3. **result = simulator.run(compiled_circuit, shots=num_days).result()**
    - The simulator runs the compiled quantum circuit.
    - shots=num_days specifies the number of times the circuit should be executed (e.g., if num_days is 100, the circuit is run 100 times).
    - The result() method collects the outcomes of all the runs.
4. **counts = result.get_counts()**
    - This extracts the counts of each possible measurement outcome (e.g., how many times the circuit resulted in 00, 01, 10, or 11).
    - The result is a dictionary where keys are the outcomes (bitstrings) and values are their respective counts.

```
35    percentage_changes = []
36    for outcome, frequency in counts.items():
37        decimal_value = int(outcome, 2)
38        percent_change = (decimal_value / (2**num_qubits - 1)) * 200 - 100
39        percentage_changes.append(round(percent_change, 2))
40
```

This code converts the quantum measurement outcomes into percentage changes to interpret the results in a meaningful way, scaled between -100% and 100%. Each percentage change reflects how the outcome relates to the total possible outcomes.

Line 35 – 39:

Code Explanation

1. **percentage_changes = []**
    - An empty list is created to store the percentage changes calculated for each outcome.
2. **for outcome, frequency in counts.items():**
    - Iterates over each outcome and its frequency in the counts dictionary.
    - counts is a dictionary where the keys are bitstrings (like '00', '01') and the values are the number of times those outcomes occurred.
3. **decimal_value = int(outcome, 2)**
    - Converts the outcome (a binary string) into a decimal (integer) value.
    - For example, if the outcome is '10', it becomes 2.
4. **percent_change = (decimal_value / (2**num_qubits - 1)) * 200 - 100**
    - Calculates the percentage change based on the decimal value of the outcome.

- **Formula breakdown**:
  - 2**num_qubits gives the total number of possible outcomes for the given number of qubits.
  - Subtracting 1 normalizes the decimal value to the range [0, 2**num_qubits - 1].
  - Multiplying by 200 and subtracting 100 scales the result to a percentage range of [-100, 100].

5. **percentage_changes.append(round(percent_change, 2))**
   - Rounds the calculated percentage change to two decimal places and appends it to the percentage_changes list.

Quantum Circuit With All Gates

```
1  import matplotlib.pyplot as plt
2  from qiskit import QuantumCircuit, transpile
3  from qiskit_aer import AerSimulator
4  from qiskit.visualization import plot_histogram, circuit_drawer
5
6
```

This code snippet imports libraries and specific functions/modules needed for quantum circuit design, simulation, and visualization.

Line 1 – 4:

Code Breakdown

1. **import matplotlib.pyplot as plt**
   - Imports the matplotlib.pyplot module and gives it the alias plt.
   - This module is used to create visualizations, such as graphs and charts, in Python.
   - It's often used for plotting results from quantum simulations or other computations.
2. **from qiskit import QuantumCircuit, transpile**
   - Imports two core components from Qiskit:
     - **QuantumCircuit**: Used to create and manipulate quantum circuits.
     - **transpile**: Optimizes a quantum circuit for a specific quantum computer or simulator.
3. **from qiskit_aer import AerSimulator**

- Imports the AerSimulator class from the Qiskit Aer module.
- **AerSimulator**: A simulator that mimics the behavior of a quantum computer for running quantum circuits without actual hardware.

4. **from qiskit.visualization import plot_histogram, circuit_drawer**
   - Imports visualization tools from Qiskit:
     - **plot_histogram**: Creates a histogram of quantum circuit results, often used to visualize measurement outcomes.
     - **circuit_drawer**: Draws a visual representation of the quantum circuit for better understanding and debugging.

```
13  gate_colors = {
14      "h": "#FFA500",  # Hadamard: Orange
15      "cx": "#00BFFF",  # CNOT: Deep Sky Blue
16      "x": "#FF4500",  # Pauli-X: Red
17      "y": "#8A2BE2",  # Pauli-Y: Purple
18      "z": "#32CD32",  # Pauli-Z: Lime Green
19      "measure": "#FFD700"  # Measure: Gold
20  }
21
```

This snippet defines a dictionary named gate_colors that maps quantum gates to specific colors. The colors are represented in hexadecimal format (commonly used in web design and graphics).

Line 13 – 19:

Code Breakdown

1. **gate_colors = { ... }**
   - The dictionary gate_colors assigns each quantum gate a unique color for visualization purposes.
   - Each key is the name of a quantum gate, and each value is a string representing a color in hexadecimal format.
2. **Mapping of Gates to Colors:**
   - **"h": "#FFA500"**: Assigns the Hadamard gate (h) the color **orange**.
   - **"cx": "#00BFFF"**: Assigns the Controlled NOT (CNOT) gate (cx) the color **deep sky blue**.
   - **"x": "#FF4500"**: Assigns the Pauli-X gate (x) the color **red**.
   - **"y": "#8A2BE2"**: Assigns the Pauli-Y gate (y) the color **purple**.
   - **"z": "#32CD32"**: Assigns the Pauli-Z gate (z) the color **lime green**.

- **"measure": "#FFD700"**: Assigns the measurement operation (measure) the color **gold**.

```
23    num_qubits = 9
24    circuit = QuantumCircuit(num_qubits)
25
```

This code snippet initializes a quantum circuit with a specified number of qubits.

Line 23 – 24:

Code Breakdown

1. **num_qubits = 9**
   - This sets the variable num_qubits to 9.
   - It defines the number of qubits that will be used in the quantum circuit.
2. **circuit = QuantumCircuit(num_qubits)**
   - This creates a new quantum circuit with num_qubits (in this case, 9 qubits) using the QuantumCircuit class from Qiskit.
   - The circuit variable now holds the quantum circuit object, which can be used to add gates, operations, or measurements.

```
27    circuit.h(range(num_qubits))  # Hadamard gates
28    circuit.cx( control_qubit: 0,  target_qubit: 1)  # CNOT gate
29    circuit.x(2)  # Pauli-X gate
30    circuit.y(3)  # Pauli-Y gate
31    circuit.z(4)  # Pauli-Z gate
32    circuit.measure_all()  # Measurement gates
33
```

This snippet demonstrates how quantum gates and measurements are applied to a quantum circuit with multiple qubits.

Line 27 – 32:

Code Breakdown

1. **circuit.h(range(num_qubits))**
   - Applies Hadamard gates to all qubits in the circuit.
   - **Purpose**: Creates superposition for each qubit, preparing them for further quantum operations.

2. **circuit.cx(0, 1)**
    - Applies a CNOT (Controlled NOT) gate with qubit 0 as the control qubit and qubit 1 as the target qubit.
    - **Purpose**: Entangles qubit 0 and qubit 1, ensuring their states are correlated. This is a key operation in many quantum algorithms.
3. **circuit.x(2)**
    - Applies a Pauli-X gate to qubit 2.
    - **Purpose**: Flips the state of qubit 2 from $|0\rangle$ to $|1\rangle$ or vice versa, similar to a classical NOT operation.
4. **circuit.y(3)**
    - Applies a Pauli-Y gate to qubit 3.
    - **Purpose**: Rotates the state of qubit 3 around the Y-axis of the Bloch sphere, introducing phase and state changes.
5. **circuit.z(4)**
    - Applies a Pauli-Z gate to qubit 4.
    - **Purpose**: Flips the phase of qubit 4's state around the Z-axis, adding a phase shift without changing the probability amplitudes.
6. **circuit.measure_all()**
    - Measures all qubits in the circuit.
    - **Purpose**: Collapses the quantum states of the qubits into classical bits, enabling the retrieval of the computational result.

```python
print("Displaying and saving quantum circuit with all gates...")
fig_circuit, ax_circuit = plt.subplots(figsize=(12, 8))
ax_circuit.axis("off")  # Remove axes for better display
circuit_drawer(
    circuit,
    output="mpl",
    style={"displaycolor": gate_colors},
    ax=ax_circuit
)
plt.title("Quantum Circuit With All Gates")
plt.savefig("Quantum_Circuit_With_All_Gates.png")
plt.show()  # Ensure the circuit displays
```

This code snippet visualizes a quantum circuit with custom styling, saves it as an image file, and displays it.

Line 35 – 46:

Code Breakdown

1. **print("Displaying and saving quantum circuit with all gates...")**
    - Displays a message in the console to indicate that the circuit is being visualized and saved.
2. **fig_circuit, ax_circuit = plt.subplots(figsize=(12, 8))**
    - Creates a figure (fig_circuit) and an axis (ax_circuit) for plotting the quantum circuit using Matplotlib.
    - **figsize=(12, 8)** sets the size of the plot to be 12 inches wide and 8 inches tall.
3. **ax_circuit.axis('off')**
    - Removes the axes (gridlines, ticks) from the plot for a cleaner display.
4. **circuit_drawer(...)**
    - Visualizes the quantum circuit using the circuit_drawer function from Qiskit.
    - **Parameters:**
        - circuit: The quantum circuit to be visualized.
        - output="mpl": Specifies that the visualization should be rendered using Matplotlib.
        - style={"displaycolor": gate_colors}: Applies custom gate colors defined in the gate_colors dictionary.
        - ax=ax_circuit: Draws the circuit on the axis created earlier.
5. **plt.title("Quantum Circuit With All Gates")**
    - Adds a title to the visualization for clarity.
6. **plt.savefig("Quantum_Circuit_With_All_Gates.png")**
    - Saves the visualization as an image file named Quantum_Circuit_With_All_Gates.png.
7. **plt.show()**
    - Displays the quantum circuit visualization in the output (e.g., in a Jupyter notebook or Python script window).

```
49      simulator = AerSimulator()
50      compiled_circuit = transpile(circuit, simulator)
51      result = simulator.run(compiled_circuit, shots=1000).result()
52      counts = result.get_counts()
53
54      print("Displaying and saving histogram of quantum results...")
55      plot_histogram(counts, color="#8A2BE2")  # Using Pauli-Y color for consistency
56      plt.title("Quantum Histogram With All Gates")
57      plt.savefig("Quantum_Histogram_With_All_Gates.png")
58      plt.show()  # Ensure the histogram displays
59
60
61  if __name__ == "__main__":
62      # Call the function to visualize, save, and display
63      visualize_and_save_gates_with_display()
64
```

This code snippet simulates a quantum circuit, visualizes the results as a histogram, and saves the histogram as an image.

Line 27 – 32:

Code Breakdown

1. **Simulation Setup:**
   - **simulator = AerSimulator()**: Initializes a quantum simulator to simulate the circuit.
   - **compiled_circuit = transpile(circuit, simulator)**: Optimizes (or compiles) the quantum circuit for the simulator to improve performance and compatibility.
   - **result = simulator.run(compiled_circuit, shots=1000).result()**: Runs the compiled circuit on the simulator with 1,000 executions (shots) and stores the results.
   - **counts = result.get_counts()**: Retrieves the counts (frequencies) of each measurement outcome, where the keys are bitstrings (e.g., "000" or "101") and the values are how often those outcomes occurred.

2. **Histogram Visualization:**
   - **print("Displaying and saving histogram of quantum results...")**: Prints a message to indicate that the histogram is being generated and saved.
   - **plot_histogram(counts, color="#8A2BE2")**: Visualizes the measurement results as a histogram. The color="#8A2BE2" sets the histogram color (purple, matching the Pauli-Y gate color for consistency).
   - **plt.title("Quantum Histogram With All Gates")**: Adds a title to the histogram.
   - **plt.savefig("Quantum_Histogram_With_All_Gates.png")**: Saves the histogram as an image file with the specified filename.

- **plt.show()**: Displays the histogram in the output (e.g., in a Jupyter notebook or Python script window).

3. **Entry Point for Execution:**
   - **if __name__ == "__main__":**: Ensures that the code within this block is executed only if the script is run as the main program, not when it's imported as a module.
   - **visualize_and_save_gates_with_display()**: Calls a function (defined elsewhere) to visualize, save, and display the quantum circuit and results.